

eBPF



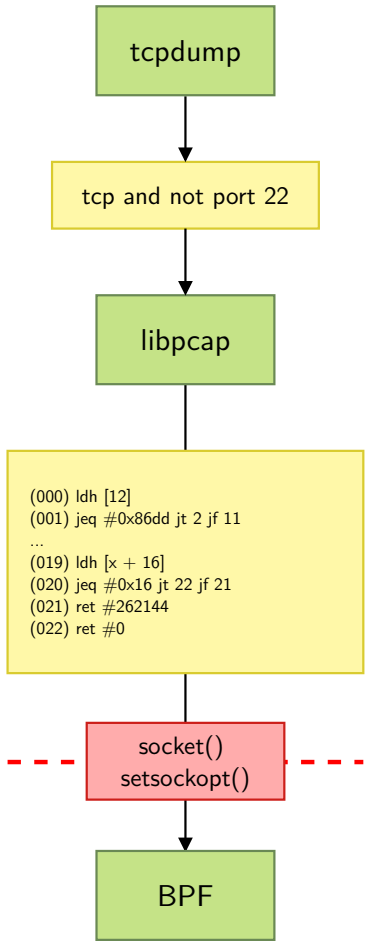
The ancestor: Berkeley Packet filter

- BPF stands for Berkeley Packet Filter and was initially used for network packet filtering
- BPF is implemented and used in Linux to perform Linux Socket Filtering (see networking/filter)
- tcpdump and Wireshark heavily rely on BPF (through libpcap) for packet capture



BPF in libpcap: setup

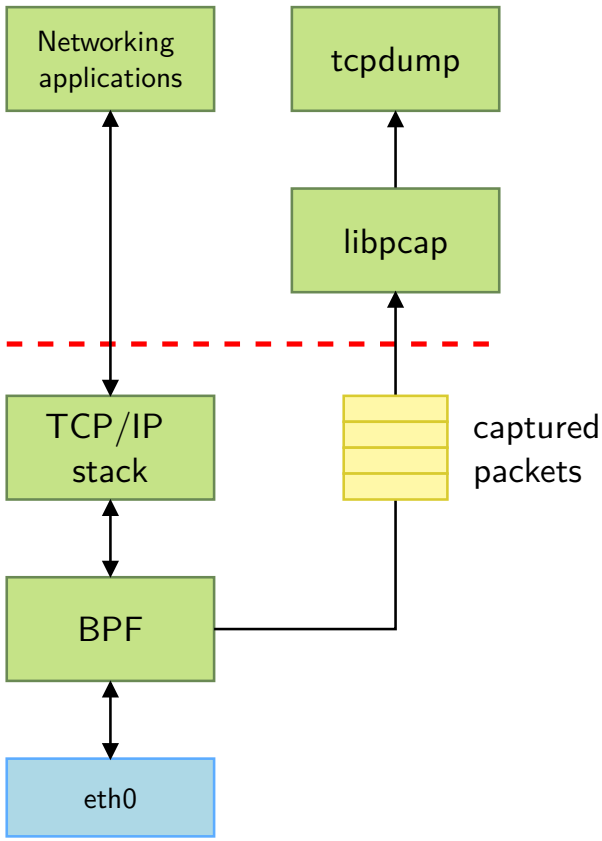
- tcpdump passes the capture filter string from the user to libpcap
- libpcap translates the capture filter into a binary program
  - This program uses the instruction set of an abstract machine (the "BPF instruction set")
- libpcap sends the binary program to the kernel via the `setsockopt()` syscall





BPF in libpcap: capture

- The kernel implements the BPF “virtual machine”
- The BPF virtual machine executes the program for every packet
- The program inspects the packet data and returns a non-zero value if the packet must be captured
- If the return value is non-zero, the packet is captured in addition to regular packet processing





eBPF (1/2)

- eBPF is a new framework allowing to run small user programs directly in the kernel, in a safe and efficient way. It has been added in kernel 3.18 but it is still evolving and receiving updates frequently.
- eBPF programs can capture and expose kernel data to userspace, and also alter kernel behavior based on some user-defined rules.
- eBPF is event-driven: an eBPF program is triggered and executed on a specific kernel event
- A major benefit from eBPF is the possibility to reprogram the kernel behavior, without performing kernel development:
  - no risk of crashing the kernel because of bugs
  - faster development cycles to get a new feature ready



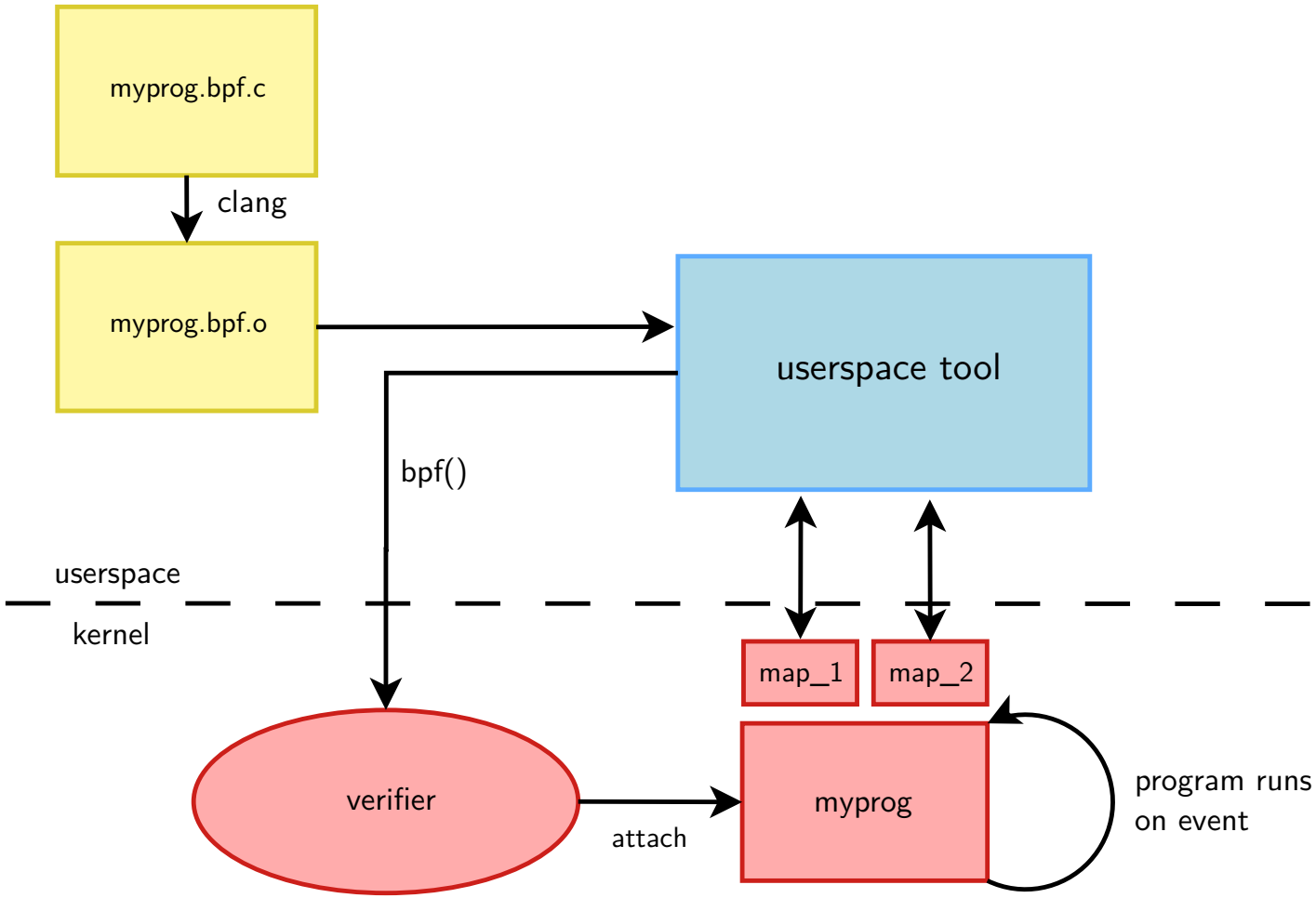
Image credits: https://ebpf.io/



eBPF (2/2)

- The most notable eBPF features are:
  - A new instruction set, interpreter and verifier
  - A wide variety of “attach” locations, allowing to hook programs almost anywhere in the kernel
  - dedicated data structures called “maps”, to exchange data between multiple eBPF programs or between programs and userspace
  - A dedicated `bpf()` syscall to manipulate eBPF programs and data
  - plenty of (kernel) helper functions accessible from eBPF programs.

eBPF program lifecycle





Kernel configuration for eBPF

- CONFIG_NET to enable eBPF subsystem
- CONFIG_BPF_SYSCALL to enable the `bpf()` syscall
- CONFIG_BPF_JIT to enable JIT on programs and so increase performance
- CONFIG_BPF_JIT_ALWAYS_ON to force JIT
- CONFIG_BPF_UNPRIV_DEFAULT_OFF=n in **development** to allow eBPF usage without root
- You may then want to enable more general features to "unlock" specific hooking locations:
  - CONFIG_KPROBES to allow hooking programs on kprobes
  - CONFIG_TRACING to allow hooking programs on kernel tracepoints
  - CONFIG_NET_CLS_BPF to write packets classifiers
  - CONFIG_CGROUP_BPF to attach programs on cgroups hooks



eBPF ISA

- eBPF is a “virtual” ISA, defining its own set of instructions: load and store instructions, arithmetic instructions, jump instructions,etc
- It also defines a set of 10 64-bits wide registers as well as a calling convention:
  - `R0`: return value from functions and BPF program
  - `R1, R2, R3, R4, R5`: function arguments
  - `R6, R7, R8, R9`: callee-saved registers
  - `R10`: stack pointer

```
; bpf_printk("Hello %s\n", "World");
      0:  r1 = 0x0 ll
      2:  r2 = 0xa
      3:  r3 = 0x0 ll
      5:  call 0x6
; return 0;
      6:  r0 = 0x0
      7:  exit
```



The eBPF verifier

- When loaded into the kernel, a program must first be validated by the eBPF verifier.
- The verifier is a complex piece of software which checks eBPF programs against a set of rules to ensure that running those may not compromise the whole kernel. For example:
  - a program must always return and so not contain paths which could make them "infinite" (e.g: no infinite loop)
  - a program must make sure that a pointer is valid before dereferencing it
  - a program cannot access arbitrary memory addresses, it must use passed context and available helpers
- If a program violates one of the verifier rules, it will be rejected.
- Despite the presence of the verifier, you still need to be careful when writing programs! eBPF programs run with preemption enabled (but CPU migration disabled), so they can still suffer from concurrency issues
  - There are mechanisms and helpers to avoid those issues, like per-CPU maps types.



Program types and attach points

- There are different categories of hooks to which a program can be attached:
  - an arbitrary kprobe
  - a kernel-defined static tracepoint
  - a specific perf event
  - throughout the network stack
  - an arbitrary uprobe
  - and a lot more, see bpf_attach_type
- A specific attach-point type can only be hooked with a set of specific program types, see bpf_prog_type and bpf/libbpf/program_types.
- The program type then defines the data passed to an eBPF program as input when it is invoked. For example:
  - A `BPF_PROG_TYPE_TRACEPOINT` program will receive a structure containing all data returned to userspace by the targeted tracepoint.
  - A `BPF_PROG_TYPE_SCHED_CLS` program (used to implement packet classifiers) will receive a struct ___sk_buff, the kernel representation of a socket buffer.
  - You can learn about the context passed to any program type by checking include/linux/bpf_types.h



eBPF maps

- eBPF programs exchange data with userspace or other programs through maps of different natures:
  - `BPF_MAP_TYPE_ARRAY`: generic array storage. Can be differentiated per CPU
  - `BPF_MAP_TYPE_HASH`: a storage composed of key-value pairs. Keys can be of different types: `__u32`, a device type, an IP address...
  - `BPF_MAP_TYPE_QUEUE`: a FIFO-type queue
  - `BPF_MAP_TYPE_CGROUP_STORAGE`: a specific hash map keyed by a cgroup id. There are other types of maps specific to other object types (inodes, tasks, sockets, etc)
  - etc...
- For basic data, it is easier and more efficient to directly use eBPF global variables (no syscalls involved, contrary to maps)



The `bpf()` syscall

- The kernel exposes a `bpf()` syscall to allow interacting with the eBPF subsystem
- The syscall takes a set of subcommands, and depending on the subcommand, some specific data:
  - BPF_PROG_LOAD to load a bpf program
  - BPF_MAP_CREATE to allocate maps to be used by a program
  - BPF_MAP_LOOKUP_ELEM to search for an entry in a map
  - BPF_MAP_UPDATE_ELEM to update an entry in a map
  - etc
- The syscall works with file descriptors pointing to eBPF resources. Those resources (program, maps, links, etc) remain valid while there is at least one program holding a valid file descriptor to it. Those are automatically cleaned once there are no user left.
- For more details, see man 2 bpf



Writing eBPF programs

- eBPF programs can either be written directly in raw eBPF assembly or in higher level languages (e.g: C or rust), and are compiled using the clang compiler.
- The kernel provides some helpers that can be called from an eBPF program:
  - `bpf_trace_printk` Emits a log to the trace buffer
  - `bpf_map_lookup,update,delete` *elem Manipulates maps*
  - `bpf_probe_read,write`[*user*] Safely read/write data from/to kernel or userspace
  - `bpf_get_current_pid_tgid` Returns current Process ID and Thread group ID
  - `bpf_get_current_uid_gid` Returns current User ID and Group ID
  - `bpf_get_current_comm` Returns the name of the executable running in the current task
  - `bpf_get_current_task` Returns the current struct task_struct
  - Many other helpers are available, see man 7 bpf-helpers
- Kernel also exposes kfuncs (see bpf/kfuncs), but contrary to bpf-helpers, those do not belong to the kernel stable interface.



Manipulating eBPF program

- There are different ways to build, load and manipulate eBPF programs:
  - One way is to write an eBPF program, build it with clang, and then load it, attach it and read data from it with bare `bpf()` calls in a custom userspace program
  - One can also use `bpftool` on the built ebpf program to manipulate it (load, attach, read maps, etc), without writing any userspace tool
  - Or we can write our own eBPF tool thanks to some intermediate libraries which handle most of the hard work, like libbpf
  - We can also use specialized frameworks like BCC or bpftrace to really get all operations (bpf program build included) handled



BCC

- BPF Compiler Collection (BCC) is (as its name suggests) a collection of BPF based tools.
- BCC provides a large number of ready-to-use tools written in BPF.
- Also provides an interface to write, load and hook BPF programs more easily than using “raw” BPF language.
- Available on a large number of architectures and distributions (but not packaged in Buildroot)
  - On debian, when installed, all tools are named `<tool>-bpfcc`.
- BCC requires a kernel version >= 4.1.
- BCC evolves quickly, many distributions have old versions: you may need to compile from the latest sources



Image credits:
https://github.com/iovisor/bcc

BCC tools

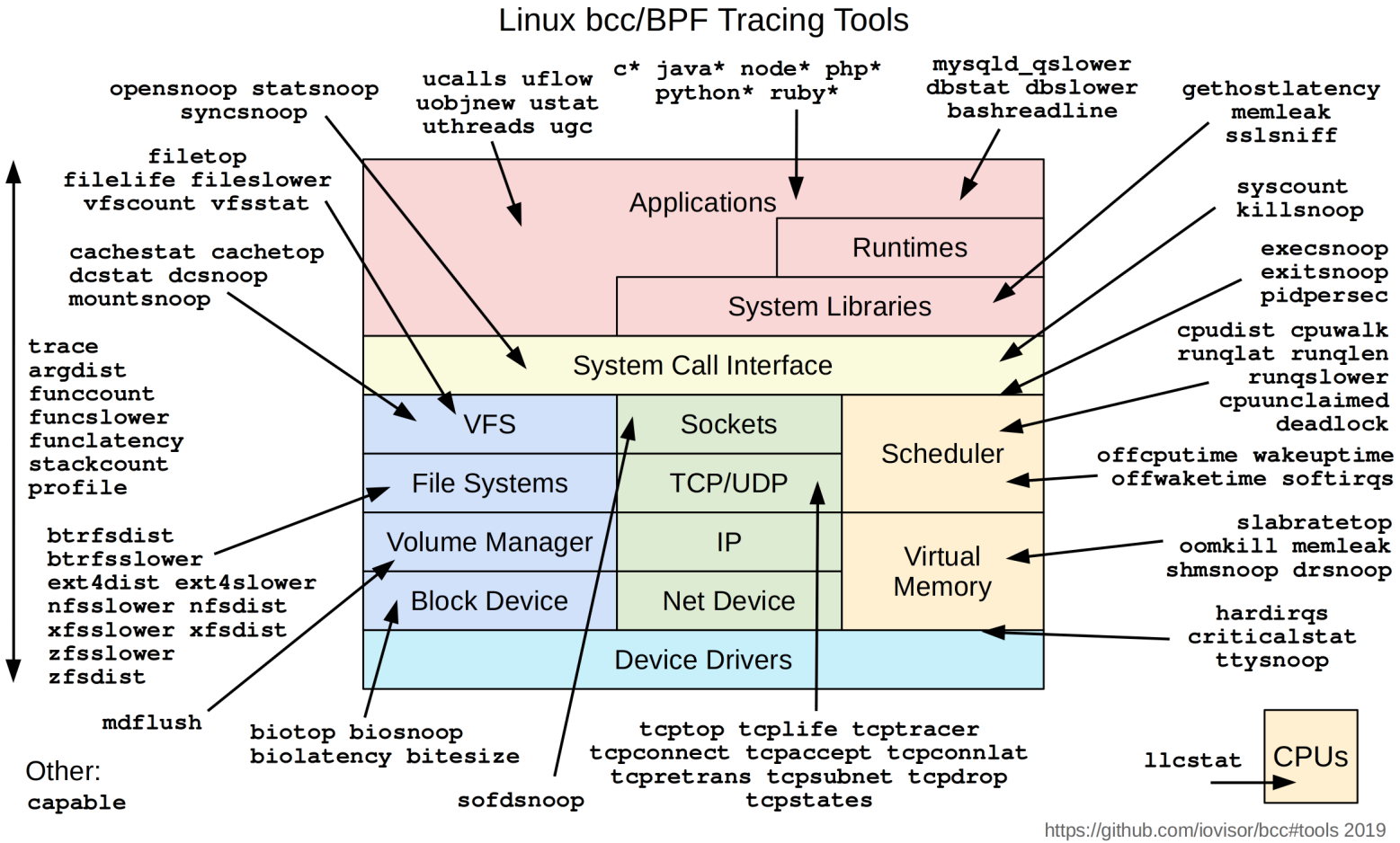


Image credits: https://www.brendangregg.com/ebpf.html



BCC Tools example

- `profile.py` is a CPU profiler allowing to capture stack traces of current execution. Its output can be used for flamegraph generation:

```
$ git clone https://github.com/brendangregg/FlameGraph.git
$ profile.py -df -F 99 10 | ./FlameGraph/flamegraph.pl > flamegraph.svg
```

- `tcpconnect.py` script displays all new TCP connections live

```
$ tcpconnect
PID    COMM         IP SADDR            DADDR            DPORT
220321 ssh          6  ::1              ::1              22
220321 ssh          4  127.0.0.1        127.0.0.1        22
17676  Chrome_Child 6  2a01:cb15:81e4:8100:37cf:d45b:d87d:d97d
2606:50c0:8003::154 443
[...]
```

- And much more to discover at https://github.com/iovisor/bcc



Using BCC with python

- BCC exposes a `bcc` module, and especially a `BPF` class
- eBPF programs are written in C and stored either in external files or directly in a python string.
- When an instance of the `BPF` class is created and fed with the program (either as string or file), it automatically builds, loads, and possibly attaches the program
- There are multiple ways to attach a program:
  - By using a proper program name prefix, depending on the targeted attach point (and so the attach step is performed automatically)
  - By explicitly calling the relevant attach method on the `BPF` instance created earlier



Using BCC with python

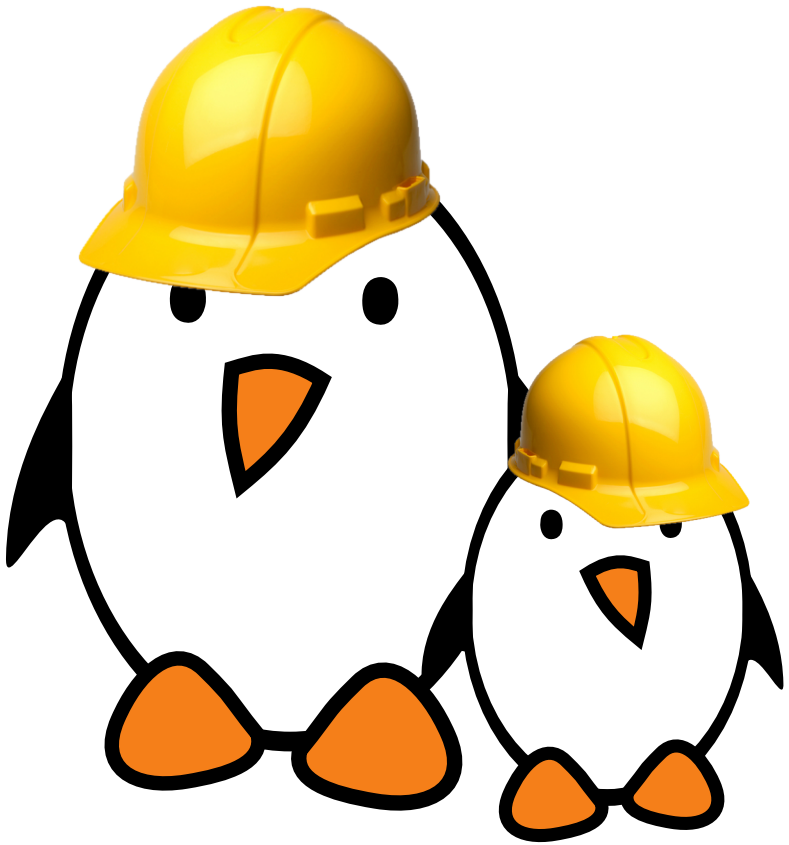
- Hook with a *kprobe* on the `clone()` system call and display `"Hello, World!"` each time it is called

```
#!/usr/bin/env python3

from bcc import BPF

# define BPF program
prog = '''
int hello(void *ctx) {
    bpf_trace_printk("Hello, World!\\n");
    return 0;
}
'''
# load BPF program
b = BPF(text=prog)
b.attach_kprobe(event=b.get_syscall_fnname("clone"), fn_name="hello")
```

Practical lab - Custom eBPF tool with BCC



- Creating custom tracing tools with BCC framework